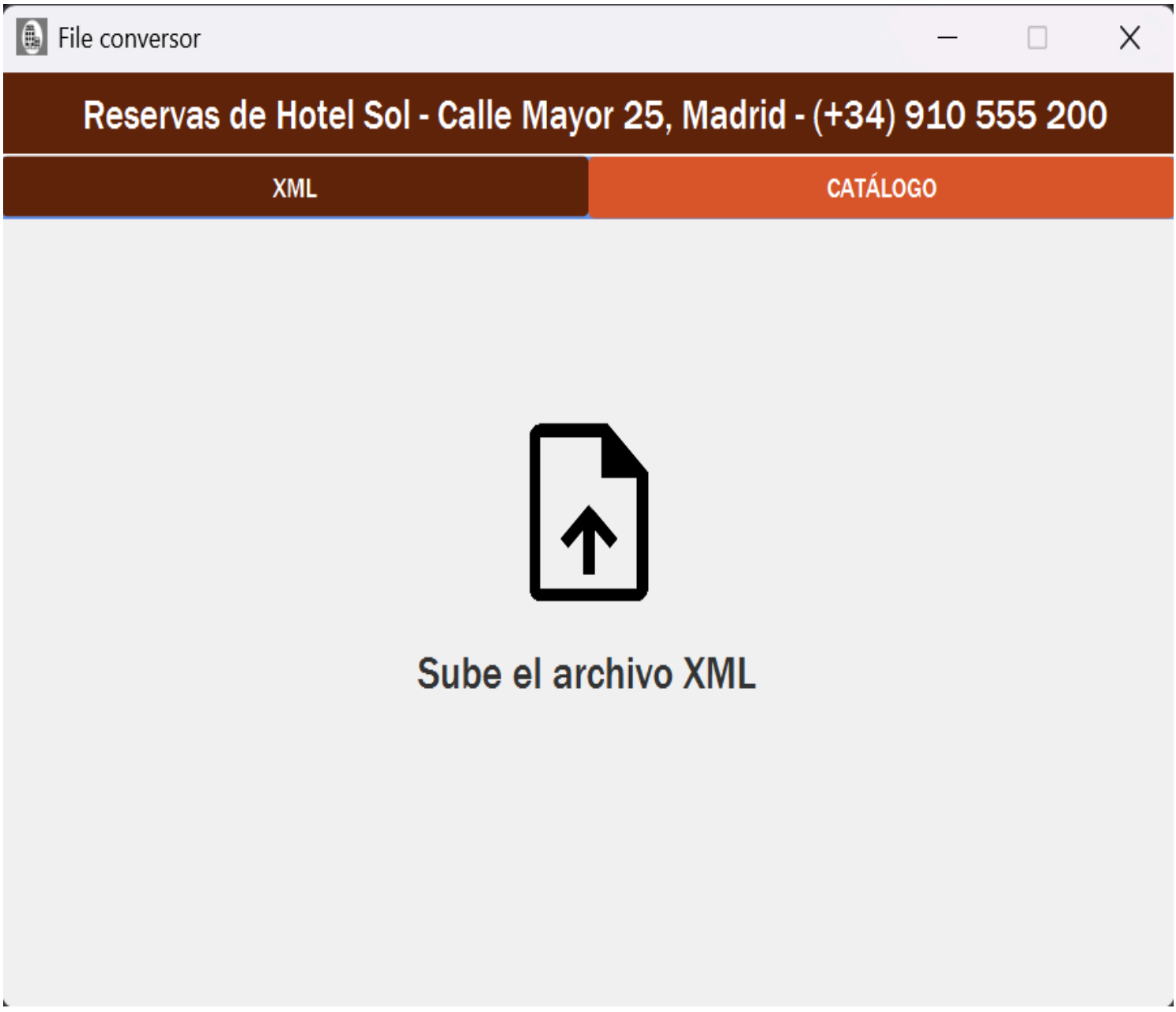
Memoria Técnica: Empaquetado y Distribución Autónoma de Aplicaciones JavaFX



Proyecto: HotelApp

Entorno de Desarrollo: IntelliJ IDEA, Maven, Java 21, JavaFX 21

Fecha: Octubre 2026

Alumnos: David López Zurrón y Rodrigo García Sánchez

1. Resumen Ejecutivo

El objetivo de este trabajo técnico es documentar el proceso completo de arquitectura, empaquetado y preparación para la distribución del proyecto JavaFX **HotelApp**. Se abordan las problemáticas de concurrencia de módulos de ejecución en JavaFX, la resolución mediante patrones de arranque (Launcher), la configuración del plugin de empaquetado sombreado (*Fat JAR / Uber-JAR*) en Maven y el despliegue final en formato ejecutable portable sin dependencias externas mediante jpackage.

2. Problemas del Entorno y Arquitectura JavaFX

Al desarrollar aplicaciones con JavaFX sobre versiones modernas de Java (JDK 11+), el sistema de módulos (JPMS) requiere una gestión estricta de los puntos de entrada. Cuando una clase extiende directamente de javafx.application.Application e implementa el método main(String[] args), intentar ejecutarla desde un único archivo JAR empaquetado produce errores en tiempo de ejecución como:

```
Error: el método principal debe devolver un valor del tipo void en la clase {0},
defina el método principal del siguiente modo: public static void main(String[]
args)
```

Este conflicto ocurre porque la Máquina Virtual de Java (JVM) comprueba los componentes del entorno de JavaFX antes de invocar la clase principal. Si dicha clase hereda de Application y está contenida en el mismo paquete/JAR sin los argumentos del módulo especificados explícitamente, la inicialización falla.

3. Solución Arquitectónica: Patrón Launcher

Para soslayar el conflicto de verificación de la JVM, se implementó una clase desacoplada como punto de entrada nativo, denominada MainLauncher.java:

```
package es.promehub;

public class MainLauncher {
    public static void main(String[] args) {
        Main.main(args);
    }
}
```

Al no heredar de Application, MainLauncher actúa como una clase de Java estándar. La JVM la ejecuta sin requerir validación previa de los módulos gráficos y, posteriormente, delega la ejecución al método Main.main(args), el cual inicia el ciclo de vida de la aplicación JavaFX de forma transparente.

4. Configuración del Proyecto Maven (`pom.xml`)

El archivo de configuración de Maven integra las dependencias necesarias (JavaFX Controls, JavaFX FXML, Jakarta JAXB) y los plugins de compilación y empaquetado. Para consolidar todas las dependencias en un único ejecutable, se configuró el maven-shade-plugin, especificando la clase MainLauncher dentro del manifiesto:

Plugin	Versión	Función en el Proyecto
maven-compiler-plugin	3.13.0	Establece el nivel de compilación objetivo en Java 21.
maven-jar-plugin	3.5.0	Genera el artefacto JAR básico del proyecto.
javafx-maven-plugin	0.0.8	Facilita la ejecución directa de la aplicación durante la fase de desarrollo.
maven-shade-plugin	3.5.0	Fusiona el código del proyecto y todas sus dependencias en un archivo *Fat JAR* (HotelApp.jar) e inyecta la clase principal en el MANIFEST.MF.

Extracto clave del `pom.xml` para el empaquetado:

```
<plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-shade-plugin</artifactId>
    <version>3.5.0</version>
    <executions>
        <execution>
            <phase>package</phase>
            <goals>
                <goal>shade</goal>
            </goals>
            <configuration>
                <transformers>
                    <transformer
implementation="org.apache.maven.plugins.shade.resource.ManifestResourceTransforme
r">
                        <mainClass>es.promehub.MainLauncher</mainClass>
                    </transformer>
                </transformers>
            </configuration>
        </execution>
    </executions>
</plugin>
```

5. Opciones de Distribución para el Usuario Final

Opción A: Ejecución mediante comandos (Requiere Java instalado)

El usuario debe contar con un Java Runtime Environment (JRE 21) instalado en el sistema operativo y ejecutar desde la consola:

```
java -jar HotelApp.jar
```

Opción B: Generación de Ejecutable Portable Nativo con `jpackage` (Recomendada)

Para evitar requerir que el usuario final tenga Java instalado en su sistema operativo, se utiliza la herramienta oficial del JDK denominada jpackage. Esta utilidad empaqueta el artefacto JAR empaquetado junto con un entorno de ejecución de Java (Runtime) privado, minimizado y dedicado.

Comando de empaquetado portable:

```
jpackage --name HotelApp --input target --main-jar HotelApp.jar --main-class es.promehub.MainLauncher --type app-image
```

Desglose detallado de parámetros del comando:

- **--name HotelApp:** Define el nombre comercial de la aplicación y asigna el nombre al archivo ejecutable nativo resultante (HotelApp.exe en sistemas Windows).
- **--input target:** Especifica el directorio de entrada donde se encuentra el archivo JAR previamente compilado y empaquetado por Maven.
- **--main-jar HotelApp.jar:** Determina el nombre exacto del archivo JAR empaquetado que contiene la aplicación y todas sus dependencias fusionadas.
- **--main-class es.promehub.MainLauncher:** Define la clase ejecutable de entrada (el Launcher desacoplado) que inicializará la aplicación.
- **--type app-image:** Especifica el tipo de salida deseado. El modo app-image construye una imagen de aplicación/carpeta autónoma e independiente (portable), evitando la necesidad de generar un instalador ejecutable (como .exe o .msi).

Resultado y Distribución: Se genera una carpeta independiente que contiene el ejecutable directo HotelApp.exe y todas sus librerías dinámicas junto con el entorno Java incrustado. Para distribuirlo a los usuarios finales, simplemente se comprime dicha carpeta en un archivo .zip. El usuario únicamente debe descomprimir el archivo y hacer doble clic sobre HotelApp.exe para ejecutar la aplicación al instante.